통합반 1회차 — 낭독 대본

[SLIDE 01] 설비 데이터에서 판단과 조치까지, 그리고 온톨로지가 필요한 이유

안녕하세요. 경남대학교 피지컬AI 사관학교에서 이 과정의 강의를 맡게 된 유엔진솔루션즈 연구원 안치윤입니다. 이 과정에서는 설비에서 나온 데이터를 지식으로 정리하고, AI의 판단과 사람의 승인을 거쳐 실제 조치로 이어지는 과정을 함께 만들어 갑니다.

공장 설비에서 기름 온도가 오르기 시작했습니다. 화면에는 온도가 정확히 나옵니다. 그런데 운전원이 알고 싶은 것은 따로 있죠. 왜 뜨거워졌고, 지금 무엇을 해야 할까요?

오늘은 이 물음에서 출발해 데이터가 판단과 조치로 이어지는 흐름을 따라가면서, 온톨로지라는 지식 모델이 왜 필요한지 알아보겠습니다.

[SLIDE 02] 학습목표

오늘 수업의 학습목표는 네 가지입니다. 배운 것을 내 말로 남겨 다시 찾아 쓰고, 설비에서 판단과 조치까지 이어지는 시스템 구조를 이해합니다. 준비된 환경에서 그 흐름을 직접 확인하고, 판단에 필요한 지식을 관계로 표현해 봅니다.

처음 보는 이름이 많아도, 각 부분이 맡은 역할을 쿨러 사건에 연결하면 함께 익힐 수 있습니다.

[SLIDE 03] 목차

수업은 화면에 보이는 다섯 단원 순서로 진행하겠습니다. 첫 단원은 학습 내용의 지식 자산화입니다. 설비의 구조와 판단 과정을 배운 뒤에도 내 말로 설명하고 다시 써 볼 수 있어야 하겠죠. 그래서 배운 개념과 실습 결과를 연결해 둘 학습노트부터 만듭니다.

두 번째는 시스템 구조와 데이터 흐름입니다. 그 노트에 정리할 내용이기도 해요. 데이터가 어디로 가는지 따라본 뒤, 환경을 실행하고 사건의 처리 과정을 확인합니다.

뒤의 세 단원은 판단에 필요한 지식을 다룹니다. 먼저 수치만으로 조치를 정할 수 없는 이유를 알아보고, 흩어진 판단 근거를 어떻게 연결할지 생각해 봅니다. 마지막에는 그 연결의 뜻을 정하는 온톨로지를 배우고 직접 관계 그림을 그려 보겠습니다.

[SLIDE 04] 학습 내용의 지식 자산화

첫 단원에서는 수업이 끝난 뒤에도 꺼내 쓸 수 있는 학습 기록을 만들려고 합니다. 수업을 듣고 나면 자료는 남죠. 그런데 며칠 뒤에 같은 문제가 생겼을 때 그 자료를 꺼내 설명하거나 해결에 쓸 수 있을까요? 자료를 모아 두는 데서 한 걸음 더 나아가, 이해한 내용을 기록으로 남기는 방법이 필요합니다.

[SLIDE 05] AI가 정리한 자료와 내 머릿속의 이해

요즘은 필기와 강의 자료를 AI에게 넘기면, 보기 좋게 정리된 문서가 금방 나오죠. 그런데 그 문서는 읽기 쉬운 형태로 바뀌었을 뿐이에요. 내용이 내 머릿속에 들어온 건 아닙니다.

더 큰 문제는 착각이에요. 깔끔하게 정리된 문서를 읽고 있으면, 그 내용을 내가 안다고 느끼게 됩니다. 문장이 술술 읽히니까요. 그런데 자료를 덮고 설명해 보려고 하면 말이 막혀요. 읽을 수 있는 것과 설명할 수 있는 것은 다르거든요.

오른쪽은 반대 방향입니다. 자료를 덮고, 이해한 만큼 백지에 직접 써 보는 거예요. 형식은 신경 쓰지 않아도 됩니다. 쓰다 보면 막히는 곳이 나오는데, 바로 거기가 내가 모르는 부분이에요. 그렇게 직접 쓰고 고친 내용이 머릿속에 남습니다.

왜 직접 써 본 내용이 더 잘 남을까요? 읽을 때는 문장이 눈앞에 있으니 알아보기만 하면 됩니다. 써 볼 때는 아무것도 없는 데서 내용을 꺼내야 하죠. 이 꺼내는 일이 곧 설명하는 일이고, 꺼내지지 않는 곳에서 내가 모르는 부분이 드러나요. 알아보는 것과 꺼내는 것은 다른 능력입니다. 남에게 설명하거나 문제를 풀 때 필요한 건 꺼내는 쪽이고요. 그래서 AI가 정리한 문서는, 먼저 내가 꺼내 써 본 뒤에 내 글과 견주어 보는 참고 자료로 씁니다.

[SLIDE 06] 백지에 넓게 써 보고, 너비부터 고친다

백지에 쓰려고 하면 어디부터 시작할지 막막할 수 있어요. 처음부터 완벽한 글을 만들려고 하지 말고, 이해한 내용을 넓게 훑어 보세요. 타이핑해도 되고 말로 설명해서 받아 적어도 됩니다.

쓰고 나면 세 가지를 살펴봅니다. 틀린 곳, 뜻이 흐릿한 곳, 더 쓰지 못한 곳이에요.

틀린 문장은 술술 써지기도 합니다. 잘못 알고 있어도 자신 있게 설명할 수 있으니까요. 그래서 쓴 글을 강의 자료와 출처에 대어 확인해야 합니다. 사실과 다른 문장이 있으면 먼저 바로잡아요.

말은 길게 이어지는데 무엇을 설명했는지 분명하지 않다면, 두 번째 칸에 해당합니다. 그 문장에서 주어가 무엇이고, 그 대상이 무엇을 한다는 것인지 짚어 보세요. 주어조차 정하기 어렵다면 무엇을 설명하는지 아직 분명하지 않은 거예요.

세 번째는 멈춘 자리예요. 여기서는 무엇이 빠졌는지 비교적 잘 드러납니다. 왜 그런지 설명하다가 막혔다면 바로 그 까닭을 더 공부하면 되죠.

순서는 왼쪽에서 오른쪽입니다. 잘못되거나 흐릿한 설명을 먼저 고쳐 전체의 줄기를 잡습니다. 그다음 막힌 곳을 하나씩 고릅니다. 처음부터 한 부분에만 매달리면 다른 설명이 틀렸다는 사실을 놓칠 수 있어요. AI에게 글을 검토받더라도, 고친 문장이 맞는지는 출처에서 확인합니다.

[SLIDE 07] 모르는 말은 바닥까지 내려가 판다

막힌 곳에서 모르는 용어 하나를 골랐다고 해 볼게요. 뜻을 물었는데 설명에 또 낯선 말이 나옵니다. 그 말도 모르는 채 처음 정의를 외우면, 설명은 여전히 막혀 있겠죠.

이번에는 설명에 나온 낯선 말의 뜻을 다시 묻습니다. 너무 어렵다면 이미 아는 말로 풀어 달라고 해도 됩니다. 이렇게 내려가다가 설명을 내 말로 할 수 있는 지점에 닿으면, 거기서부터 올라옵니다.

방금 이해한 말로 그 위의 설명을 풀고, 다시 그 설명으로 처음 용어를 풀어요. 화면의 마지막 화살표가 이 과정입니다. 답을 여러 개 모으는 것보다, 그 답을 연결해서 처음의 용어를 설명할 수 있게 되는 것이 중요해요.

AI는 질문할 때마다 설명을 얻는 데 쓸 수 있습니다. 하지만 설명을 이어 붙이고, 원래 공부하던 문제에 맞는지 확인하는 일은 직접 해야 해요. 공부 범위를 벗어난 질문이 계속 생기면 따로 노트에 남겨 둡니다. 지금 풀던 용어로 돌아와 필요한 만큼 연결하는 것이 목적입니다.

[SLIDE 08] 뜻을 내 말로 옮겨 학습노트에 쌓는다

낯선 용어를 풀어 이해했다면, 그 뜻을 내 문장으로 써 봅니다. 정식 용어는 정확히 쓰되, 설명까지 원문과 똑같을 필요는 없어요.

왼쪽은 정의 문장을 기억하는 데 그친 경우입니다. 다른 말로 물으면 외운 문장부터 다시 꺼내게 돼요. 오른쪽은 원리는 지키면서 뜻을 풀어 쓴 경우입니다. 표현이 달라져도 무엇을 설명하는지 알기 때문에 자기 말로 답할 수 있죠.

고친 문장을 원래 정의와 한번 견줘 보세요. 쉽게 쓰다가 조건을 빼지는 않았는지 봅니다. 특히 원문에 없던 '항상'이나 '모든'이라는 말이 들어가면 뜻이 넓어질 수 있어요. 설명은 쉬워져도 적용되는 조건은 남아 있어야 합니다.

확인한 뒤에는 자료를 덮고 다시 설명합니다. 아직 막히면 그 부분을 보완하고, 설명할 수 있으면 학습노트에 남겨요. 배운 내용을 기록하고 연결해서 필요할 때 다시 쓰게 만드는 일이 지식 자산화입니다.

[SLIDE 09] 다시 찾아 쓰는 학습 기록의 연결

내 말로 쓴 개념을 실습에서 써 봤는데, 노트에는 '완료'라고만 적었다고 해 볼게요. 비슷한 문제가 다시 생겼을 때 무엇을 바꿨는지, 왜 그 방법을 썼는지 알 수 있을까요? 성공했다는 사실만으로는 다시 하기 어렵습니다.

화면의 세 종류의 기록을 연결하면 그 길을 남길 수 있어요. 개념 기록에는 무엇을 배웠고 왜 그렇게 되는지를 내 말로 씁니다. 참고 자료는 그 설명을 확인할 출처예요. 문서 전체만 저장하지 말고, 확인했던 대목으로 돌아갈 수 있게 연결합니다.

실습 기록에는 한 일과 나온 결과, 막힌 점과 해결한 방법을 남깁니다. 그리고 왜 그렇게 했는지 설명하는 개념 기록을 붙여요. 다음에 같은 문제를 만나면 실습 기록에서 출발해 개념과 원문까지 돌아갈 수 있습니다.

마지막 행은 이 기록을 계속 쓰는 방법입니다. 분야로 찾을 수도 있고, 이해도가 낮은 기록을 먼저 모아 볼 수도 있어요. 다시 설명해 보니 이제 이해했다면 기록과 이해도를 함께 고칩니다.

예전에 성공한 방법이라도 지금 조건이 같은지 확인해야 합니다. 결과만 적었다면 그 비교가 어렵죠. 과정과 까닭, 출처가 연결돼 있으면 어느 조건이 달라졌는지 찾아볼 수 있습니다. 이렇게 연결해 둔 학습노트는 비슷한 문제를 만났을 때 다시 꺼내 쓸 수 있습니다.

[SLIDE 10~11] 실습: 지식 자산화를 위한 개인 학습노트 만들기

첫 실습에서는 Notion으로 개념과 출처, 실습 기록이 연결되는 개인 학습노트를 만듭니다. 먼저 과정 홈과 기록을 나누는 기준을 살펴보고 직접 작성할 거예요.

과정 홈은 전체 기록의 입구입니다. 그 아래에 개념 정리, 참고 자료, 실습 결과를 기록할 곳을 마련합니다. 학습노트에는 분야와 이해도, 작성일을 붙여 필요한 기록을 찾을 수 있게 해요.

틀만 만들고 끝내지는 않습니다. 개념 하나를 내 말로 쓰고 참고 자료를 연결해 첫 기록을 완성합니다. 리뷰에서는 설명과 출처가 맞는지, 나중에 다시 찾아 쓸 수 있는지를 봅니다. 마지막에는 완성 노트에서 기록을 찾고 원문을 여는 과정을 함께 확인하겠습니다.

[SLIDE 12] HYD 시스템 구조와 데이터 흐름

두 번째 단원은 HYD 시스템의 구조와 데이터 흐름입니다. 유온 값은 어디서 재고, 어떻게 전달하며, 무엇으로 이상을 알아챌까요? 승인된 조치는 어떤 길로 설비에 돌아올까요? Notion으로 마련한 학습노트에 각 부분의 역할과 연결을 내 말로 정리하면서, 유온 값 하나가 판단과 조치로 이어지는 길을 따라가 보겠습니다.

[SLIDE 13] 설비·센서·PLC·관제의 역할

기름 온도를 화면에서 보려면 무엇이 필요할까요? 먼저 기름이 흐르는 설비가 있어야 하고, 온도를 재는 센서가 있어야 합니다. 잰 값을 보고 실제 제어를 하는 장치와, 사람이 보는 화면도 필요해요.

설비는 압력과 온도, 진동 같은 물리량이 생기는 대상입니다. 센서는 그 물리량을 전기 신호로 바꿔 계측값을 내놓는 장치예요. 유온 센서라면 기름의 온도를 값으로 알려 줍니다.

PLC는 그 값을 읽고 제어 출력을 정하는 산업용 제어기입니다. 팬을 어느 속도로 돌릴지 같은 설정을 실제 설비에 반영해요. 센서가 재는 일과 PLC가 제어하는 일은 이렇게 나뉩니다.

SCADA는 설비 데이터를 모아 감시하고 제어하는 시스템입니다. 운전원은 그 안의 화면에서 상태를 보고 조작합니다. 사람이 화면을 보고 명령을 내리는 동안에도, 설비 옆의 PLC는 계속 계측값을 읽고 제어합니다.

이 네 역할을 유압 설비 한 대에 놓아 보겠습니다.

[SLIDE 14] 유압 파워팩 HYD-01의 구성과 유온 TS1의 계측 지점

네 역할 가운데 설비와 센서를 먼저 보겠습니다. 이 그림은 실습에서 사용할 유압 설비예요. 화면의 정상값은 이상이 생겼을 때 무엇이 달라졌는지 비교할 출발점입니다.

왼쪽 아래 탱크에 작동유가 담겨 있습니다. 모터와 펌프가 기름을 끌어올려 압력을 만들고, 그 힘으로 오른쪽 위의 구동부를 움직입니다. 일을 하고 데워진 기름은 오일 쿨러에서 식은 뒤 탱크로 돌아옵니다. 옆의 팬이 쿨러에 바람을 보내 열을 공기로 내보내요.

온도계는 쿨러에 들어가기 직전의 기름 온도를 잽니다. 이 온도를 유온이라고 합니다. 냉각이 충분하지 않으면 이 자리에서 온도가 올라가는 모습을 보게 됩니다.

계측값에 붙인 이름이 태그예요. 유온, 펌프에서 내보내는 압력, 팬 진동에 각각 태그가 붙어 있습니다. 필요한 값을 고를 때 이 이름을 씁니다. 그림에서 각 태그가 어느 자리에 붙어 있는지 보세요.

유온이 오른다는 것은 들어오는 열과 내보내는 열의 균형이 달라졌다는 뜻입니다. 그런데 이 온도계만으로는 쿨러가 막혔는지, 팬에 문제가 있는지까지 알 수 없어요. 우선 이 값을 읽고 설비를 제어하는 장치부터 보겠습니다.

[SLIDE 15] Layer 1 · PLC의 반복 제어와 우선하는 보호

팬 속도를 바꾸려면 설비 옆의 PLC가 설정값을 받아 실제 제어에 반영해야 합니다. 그리고 설비가 위험해지면, 다른 곳의 답을 기다리지 않고 보호해야 해요.

PLC는 입력을 읽고 프로그램을 실행한 뒤 출력을 쓰는 일을 짧은 주기로 반복합니다. 이것이 스캔 주기입니다. 한 번 온도를 읽고 끝내는 것이 아니라, 설비가 움직이는 동안 계속 이 일을 합니다.

설정값은 제어할 목표값입니다. 화면에는 팬 속도와 펌프 부하의 설정값이 있어요. 지금 얼마나 뜨거운지 알려 주는 계측값과, 팬을 얼마나 돌리라고 정하는 설정값은 역할이 다릅니다.

위험 조건에서는 인터록이 우선합니다. 인터록은 설비를 안전한 상태로 보내는 보호 로직이에요. 유온이 과열 한계를 넘으면 PLC가 보호 정지를 실행합니다. 이렇게 보호 동작으로 정지한 상태를 트립이라고 합니다.

서버가 판단해 줄 때까지 기다리면 안 될까요? 통신이 끊기거나 답이 늦는 동안에도 설비는 계속 움직입니다. 그래서 보호 판단은 현장의 PLC가 끝냅니다. 위쪽에서 팬 속도를 바꾸라고 요청하더라도, 현재 설비가 그 명령을 받아도 되는지는 PLC가 다시 확인해요.

[SLIDE 16] HYD-01의 인터록 세 가지

PLC가 보호 정지에 쓰는 조건은 세 가지입니다. 온도가 너무 높은 경우, 압력이 너무 낮은 경우, 팬 진동이 너무 큰 경우예요.

온도부터 볼게요. 유온이 65도를 넘으면 보호 정지가 걸립니다. 기름이 너무 뜨거워지면 점도가 떨어져 부품의 마모가 빨라지기 때문입니다.

압력은 반대로 낮아질 때 문제입니다. 펌프의 토출 압력이 하한보다 낮으면 멈춥니다. 펌프 마모나 내부 누설 같은 원인을 의심할 수 있지만, 보호 동작 자체는 압력 조건으로 판단해요.

팬 진동은 상한 이상일 때 멈춥니다. 그대로 돌리면 팬 베어링이 손상될 수 있기 때문입니다. 화면의 세 기준은 이 설비에서 정지를 판단하는 선이에요. 온도와 진동은 높은 쪽, 압력은 낮은 쪽을 감시합니다.

셋 중 하나라도 위험 조건에 이르면 PLC가 보호 정지를 실행합니다. 원인을 상세하게 분석하기 전에 설비를 보호하는 동작입니다.

[SLIDE 17] 현장 통신의 지연·지터·결정성

PLC가 제때 판단하려면 신호도 제때 도착해야 합니다. 대부분 빨리 오다가 가끔 크게 늦는 통신을 생각해 보세요. 평균적으로는 빨라도, 바로 그 늦어진 순간에 보호 판단이 필요할 수 있습니다.

지연은 신호를 보낸 뒤 도착하기까지 걸리는 시간입니다. 지터는 그 지연이 매번 달라지는 정도예요. 결정성은 최악의 경우에도 정해진 시간 안에 전달되는 성질입니다. 현장 통신에서 평균 속도뿐 아니라 지터와 결정성을 보는 이유가 여기에 있습니다.

제때 도착하는지와 내용이 안전하게 전달되는지도 구분해서 봐야 해요. 허가받지 않은 사람이 내용을 읽지 못하게 지키는 것이 기밀성이고, 내용이 허가 없이 바뀌지 않게 지키는 것이 무결성입니다.

전통적인 Modbus에는 암호화와 인증이 없습니다. 그래서 통신 규약 하나에 모든 보호를 맡기지 않고, 연결할 수 있는 망을 나누고 명령을 검사하는 구조를 함께 둡니다. 이 구조는 데이터가 현장을 벗어날 때 살펴보겠습니다.

[SLIDE 18] Layer 2 · 원시 값에서 태그 메시지로

PLC가 제어에 쓰는 값을 운전 화면이나 분석 프로그램에서도 쓰려면, 받는 쪽이 알아볼 수 있는 형식으로 보내야 합니다. 화면 맨 위에는 설비 내부에서 쓰는 항목 이름과 값이 있어요. 이 값을 다른 프로그램에 보낼 메시지로 바꾸는 곳이 엣지 게이트웨이입니다.

가운데 태그 목록을 보세요. 엣지는 내부 항목을 정해진 태그에 대응시킵니다. 유온은 유온 태그로, 압력은 압력 태그로 옮깁니다. 소수점을 줄이는 것보다 중요한 건, 각 값이 무엇을 뜻하는지 정해진 약속에 맞춰 전달하는 일이에요.

그 아래가 보낼 메시지입니다. 값을 분류하는 주소인 토픽에는 설비와 태그 이름이 들어 있습니다. 본문에는 발행 시각과 값, 품질이 함께 들어가요. 받는 쪽은 값뿐 아니라 언제 보낸 메시지인지, 품질이 정상인지도 확인할 수 있습니다.

맨 아래는 운전 상태 메시지입니다. 원격 자동 모드로 운전 중이고 인터록은 정상이라는 정보를 보냅니다. 온도와 같은 계측값과, 원격 명령을 받을 수 있는 상태를 따로 전달하는 거예요.

엣지 게이트웨이는 현장과 위쪽 프로그램 사이의 통역사와 비슷합니다. 다만 뜻을 짐작해 바꾸지 않고 정해진 대응을 따릅니다. 이런 연결을 맡는 소프트웨어를 미들웨어라고 하며, 엣지 게이트웨이 역할을 하는 것에 EdgeX Foundry가 있습니다.

태그 목록의 냉각 효율은 센서에서 직접 읽은 온도와 달리 계산해서 얻는 값입니다. 이런 계측값을 가상 센서 값이라고 해요. 같은 형식의 메시지로 보내더라도 값의 출처는 다를 수 있습니다.

[SLIDE 19] 점대점 연결과 브로커 중개

엣지에서 만든 태그 메시지를 필요한 곳으로 보내야 합니다. 유온을 받아야 하는 곳은 운전 화면과 수집 서비스, 두 곳이에요. 앞으로 더 늘어날 수도 있습니다. 송신자와 수신자를 직접 묶으면 한쪽 변경이 다른 쪽에 번지기 때문에, 브로커를 사이에 둬서 연결 의존성을 줄여요.

왼쪽은 점대점 방식이에요. 송신자가 수신자마다 직접 연결하는 방식입니다. 엣지가 운전 화면에 한 줄, 수집 서비스에 한 줄을 각각 잇는 거예요. 그런데 수신자를 새로 하나 더하면, 송신하는 엣지 쪽의 연결도 고쳐야 합니다. 이렇게 한쪽의 변경이 다른 쪽의 변경을 요구하는 관계를 강한 결합이라고 해요.

오른쪽은 브로커 중개입니다. 브로커는 메시지를 중개하는 서버예요. 이걸 가운데 둡니다. 엣지는 브로커에만 보내고, 운전 화면과 수집 서비스는 브로커에서 받습니다. 새 수신자는 브로커에 메시지를 받겠다고 등록하면 됩니다. 구독하는 거예요. 송신 쪽은 고칠 게 없죠. 이렇게 송신자와 수신자의 직접 의존을 줄인 관계를 느슨한 결합이라고 합니다.

[SLIDE 20] MQTT와 브로커: 발행과 구독

브로커를 사이에 두고 메시지를 주고받는 규약이 MQTT입니다. 가벼운 발행·구독 규약으로, 장치에서 읽은 값을 전달하는 데 씁니다.

그림에서 먼저 받을 쪽을 보겠습니다. 운전 화면과 수집 서비스가 브로커에 받을 토픽을 등록합니다. 토픽은 메시지를 분류하고 전달할 때 쓰는 주소예요. 그림의 주소는 유압 설비의 유온을 가리킵니다.

엣지 게이트웨이가 그 토픽으로 값을 보내면 발행입니다. 보내는 쪽을 발행자라고 해요. 받을 토픽을 등록하고 메시지를 받는 쪽은 구독자입니다. 브로커는 등록된 구독자에게 같은 메시지를 각각 전달합니다.

방송국처럼 가운데서 받아 나눠 준다고 생각하면 이해하기 쉽습니다. 다만 아무에게나 보내는 것이 아니라 토픽을 구독한 곳에 보냅니다. 발행자는 운전 화면이 몇 개 있는지 몰라도 돼요. 구독자를 관리하는 일은 브로커가 맡습니다. 이 역할의 도구에는 EMQX가 있습니다.

받을 설비나 계측값을 구별하는 기준은 구독할 토픽 주소입니다.

[SLIDE 21] 토픽의 구조: 빗금으로 나눈 계층형 주소

토픽 주소는 빗금으로 나눈 각 자리에 뜻이 있습니다. 화면 맨 위 주소에는 공장, 설비, 데이터 종류, 태그 이름이 차례로 들어 있어요. 주소를 통째로 외우기보다, 구독할 대상을 바꿀 때 어느 자리를 고치는지 보겠습니다.

같은 설비의 유온 대신 냉각 효율을 받으려면 마지막 태그 이름이 달라집니다. 유온은 그대로 두고 다른 설비를 보려면 설비 자리가 달라져요. 아래 목록에서 그 두 차이를 확인할 수 있습니다.

맨 아래는 운전 상태를 보내는 주소입니다. 계측 태그 하나가 아니라 모드와 상태를 묶은 메시지이므로 종류를 나타내는 자리에서 끝납니다. 모든 주소가 무조건 같은 길이는 아니에요. 전달할 내용에 맞춰 구조를 정합니다.

이 규칙 덕분에 구독자는 어떤 설비의 어떤 데이터를 받을지 고를 수 있습니다. 이제 주소를 골랐으니, 메시지를 놓치거나 다시 받았을 때 어떻게 다루는지도 정해야 합니다.

[SLIDE 22] 발행·구독의 전달과 보존 범위

메시지를 한 번 놓친 상황을 생각해 보겠습니다. 온도는 새 값이 계속 오지만, 설비 명령은 놓쳤는지 확인해야 하죠. MQTT에서는 전달 품질과 마지막 메시지 보존 여부를 따로 정합니다.

전달 품질이 QoS입니다. 화면의 세 수준 중 0은 최대 한 번, 1은 최소 한 번, 2는 정확히 한 번 전달한다는 뜻이에요. 특히 최소 한 번인 경우에는 같은 메시지가 다시 도착할 수 있습니다. 받는 쪽에서 중복을 구별해야 하는 이유입니다.

QoS가 보장하는 것은 메시지 전달 수준입니다. PLC에서 같은 명령이 두 번 실행되지 않게 하는 일까지 대신하지는 않아요. 명령을 실행하는 쪽에서도 중복을 확인합니다.

보존 메시지는 토픽의 마지막 한 건을 새 구독자에게도 주는 기능입니다. 화면을 새로 켰을 때 현재 운전 상태를 바로 받아볼 수 있어요. 과거 이력 전체를 모으는 기능은 아닙니다.

HYD에서는 태그 값은 QoS 0으로 보내고 보존하지 않습니다. 주요 계측값은 매초 새로 와요. 운전 상태는 QoS 1로 보내고 마지막 값을 보존합니다. 명령은 QoS 1이지만 보존하지 않아요. 다시 연결했을 때 옛 명령이 새 명령처럼 전달되면 곤란하기 때문입니다.

브로커가 구독자에게 메시지를 밀어 주는 방식을 푸시라고 합니다. 지금 필요한 값을 전달하는 데 알맞죠. 지나간 값을 나중에 다시 분석하려면 따로 기록할 곳이 필요합니다.

[SLIDE 23] OT와 IT를 직접 잇지 않는 이유

지나간 값을 기록하고 분석하려면 브로커의 데이터를 서버로 옮겨야 합니다. 그런데 분석하고 저장하는 서버를 브로커에 바로 연결하지는 않아요. 외부와 연결된 쪽의 침해가 설비 사고로 번지지 않도록, 두 영역의 망을 나눠서 직접 연결을 막기 때문입니다.

왼쪽은 OT예요. 설비를 직접 감시하고 제어하는 영역입니다. 여기서는 멈추지 않는 가용성과 안전이 가장 중요해요. 잘못된 명령 하나가 곧바로 설비 사고로 이어지니까요.

설비를 지키는 판단을 현장의 PLC에 남겨 두는 것도 안전과 가용성을 지키기 위해서입니다. 분석 서버를 거치지 않으므로, 서버와 통신이 끊겨도 PLC는 보호 판단을 계속할 수 있어요.

오른쪽은 IT입니다. 업무 데이터를 다루는 영역이에요. 사무망과 인터넷에 연결돼 있어서 외부 침해에 노출됩니다. 그래서 기밀성과 무결성을 중시해요.

이렇게 우선순위가 다른 두 영역을 직접 이어 두면, IT 쪽이 뚫렸을 때 그 경로를 통해 설비까지 위험해집니다. 그래서 망 분리로 두 영역을 나누고, OT와 직접 연결되는 것을 제한해요.

[SLIDE 24] 완충 구역과 단방향 수집

망을 나눴다면, 데이터는 어떻게 IT로 넘어갈까요. OT와 IT 사이에 DMZ라는 완충 구역을 둡니다. 거기서 수집 서비스가 계측 데이터를 복제하고, 자동 명령은 별도의 관문에서 검사해요.

DMZ는 OT와 IT를 직접 잇지 않고, 중간에서 연결을 제한하는 완충 구역이에요. 수집 서비스와 명령 관문이 이 영역에 놓입니다.

수집 서비스는 OT의 브로커를 구독해서, 받은 메시지를 IT의 이벤트 로그로 옮겨요. 브로커에서 수집 서비스를 거쳐 이벤트 로그로 가는 거죠.

여기서는 데이터가 흐르는 방향이 중요합니다. 단방향 복제는 한 방향으로만 데이터를 복사하는 거예요. 수집 서비스는 OT 쪽으로 아무것도 발행하지 않습니다. 반대 방향, 그러니까 IT에서 설비로 가는 자동 명령은 이 길을 쓰지 않고 별도 경로로 전달돼요.

앞에서 Modbus 같은 전통 규약에는 보호 기능이 부족하다고 했죠. 이렇게 경로를 나누고, 방향을 한쪽으로 묶고, 명령은 따로 검사하는 구조가 그 부족한 부분을 보완합니다.

[SLIDE 25] Layer 3 · 메시지를 기록으로 옮긴다

수집 서비스가 브로커에서 받은 메시지를 이벤트 로그에 기록합니다. 화면의 위아래를 비교하면, 값은 같지만 출처를 적는 자리가 달라진 것을 볼 수 있어요.

위쪽 메시지는 설비와 태그 이름이 토픽 주소에 있고, 본문에는 시각과 값, 품질이 있습니다. 이 본문을 페이로드라고 합니다.

아래쪽은 로그에 남긴 레코드, 즉 기록 한 건입니다. 주소에 있던 설비와 태그 이름을 본문 안에 넣었어요. 원래 메시지와 떨어져 이 기록만 읽더라도 어느 설비의 어떤 값인지 알 수 있게 한 겁니다.

설비를 나타내는 키도 붙입니다. 키는 기록을 어느 파티션으로 보낼지 정할 때 쓰는 값이에요. 파티션은 로그의 구획입니다. 같은 설비의 기록을 같은 구획에 모아 순서대로 읽도록 합니다.

이렇게 기록을 여러 서비스가 함께 읽어 쓰는 중심 경로가 이벤트 백본입니다. 감지 서비스는 이 값으로 조건을 판정하고, 저장 서비스는 이력으로 모읍니다. 같은 기록을 써도 두 서비스가 항상 같은 속도로 처리할 필요는 없습니다. 로그가 각자 읽을 자리를 남겨 주기 때문입니다.

[SLIDE 26] 이벤트 로그의 기록·순서·다시 읽기

저장 서비스가 잠시 멈췄다고 해 볼게요. 그동안 감지 서비스까지 기다리면 이상을 놓칠 수 있습니다. 이벤트 로그에서는 각 서비스가 자기 위치에서 읽으므로 서로의 속도를 기다리지 않아도 됩니다.

로그에는 새 레코드를 뒤에 덧붙입니다. 이것이 추가 전용 로그예요. 기록을 쓰는 쪽은 생산자, 읽는 쪽은 소비자입니다. 수집 서비스가 생산자이고, 감지 서비스와 저장 서비스는 각각 소비자예요. 읽었다고 기록이 사라지지는 않습니다.

오프셋은 파티션 안의 레코드 위치입니다. 소비자는 어디까지 읽었는지 따로 관리합니다. 저장 서비스가 돌아오면 자기가 읽던 위치에서 이어 읽고, 감지 서비스는 자기 위치에서 계속 판정할 수 있어요.

파티션 안에서는 기록 순서가 보장됩니다. 같은 설비의 키를 사용하는 이유도 그 기록들을 같은 구획에 모으기 위해서예요. 여러 파티션의 모든 기록에 하나의 순서가 생기는 것은 아닙니다.

다시 읽으려면 기록이 남아 있어야 하겠죠. 화면의 태그 원본 로그는 하루 동안 보존하도록 돼 있습니다. 그 기간이 지나 지워진 기록이나 처음부터 로그에 도착하지 않은 메시지는 읽기 위치를 돌려도 얻을 수 없습니다.

이벤트 로그 역할을 하는 도구에는 Kafka가 있습니다. 여기서 기억할 기능은 기록을 남기고, 여러 소비자가 자기 위치에서 읽고, 남아 있는 기록을 다시 처리할 수 있다는 것입니다.

[SLIDE 27] Layer 4 · 스트림 처리: 흐르는 데이터에서 알아챈다

이벤트 로그의 기록을 읽는 첫 번째 계층이 레이어 4, 스트림 처리예요. 감지 서비스는 이상을 제때 알아차리기 위해, 이벤트 로그에 데이터가 기록되는 대로 읽어서 조건과 비교합니다.

왼쪽부터 볼게요. 배치 처리는 데이터를 모아 두었다가 묶어서 처리하는 방식이에요. 일정한 주기로 조회한다면, 이상이 생겨도 다음 처리 시점까지 기다려야 합니다. 그래서 이상 발생과 발견 사이에 지연이 생겨요.

오른쪽은 다른 방식이에요. 데이터가 기록되는 대로 하나씩 받아서 그 자리에서 조건과 비교합니다. 이렇게 끊임없이 도착하는 데이터를 그 자리에서 처리하는 방식을 스트림 처리라고 해요. 감지 서비스가 이 방식으로 일합니다.

유온이 기준을 넘는 순간부터 인터록이 걸리기까지 남은 시간은 길지 않아요. 그래서 이상 감지는 모아 두었다가 나중에 보는 게 아니라, 흐르는 데이터 위에서 바로 합니다. 스트림 처리 엔진 역할에는 Flink 같은 것이 있어요.

[SLIDE 28] 복합 이벤트 처리: 경보 조건과 유지 시간

유온이 높아졌다고 곧바로 쿨러 문제라고 할 수 있을까요? 감지 서비스는 쿨러가 열을 내보내는 정도와 온도가 계속 오르는지도 함께 봅니다.

화면 오른쪽의 조건은 세 가지입니다. 유온이 55도를 넘고, 냉각 효율이 70퍼센트보다 낮고, 유온이 오르는 중이어야 합니다. 기울기가 양수라는 것은 온도가 오르고 있다는 뜻이에요. 이 조건들이 함께 이어질 때 쿨러 성능 저하 경보를 냅니다.

여러 값의 조합과 추세, 지속 시간을 함께 보는 방식이 복합 이벤트 처리, CEP입니다. 온도 하나가 선을 넘었는지만 보는 것과 다릅니다.

그래프에서 온도가 가로선을 넘는 곳을 찾아보세요. 그 지점에 바로 경보가 울리지는 않습니다. 조건이 이어지는 파란 구간을 지나 경보가 발생해요. 조건이 계속 성립해야 하는 시간을 유지 시간이라고 합니다.

값이 잠깐 튀었다가 돌아왔는데도 경보를 내면 헛경보가 늘 수 있어요. 그래서 조건이 유지되는지 확인합니다. 그래프의 두 점은 실제 기록에서 기준을 넘은 관측과 경보가 발생한 관측입니다. 여기서는 그 사이에 간격이 있다는 점을 보면 됩니다.

[SLIDE 29] 경보의 발생·해제와 데이터 품질

경보가 울린 뒤 온도가 발생 기준 바로 아래로 내려왔습니다. 그 순간 바로 끄면 어떨까요? 온도가 경계 근처에서 오르내릴 때 경보도 계속 켜졌다 꺼질 수 있습니다.

그래서 발생과 해제에 다른 기준을 둡니다. 발생 쪽은 앞에서 본 복합 조건이고, 해제 쪽은 유온이 52도 아래로 내려가 더 이상 오르지 않는 상태입니다. 해제 조건도 유지 시간 동안 이어져야 해요.

발생의 온도 기준은 55도, 해제의 기준은 52도입니다. 이렇게 경계에 간격을 두어 상태가 자꾸 바뀌는 것을 줄이는 방식을 히스테리시스라고 합니다.

값이 들어오지 않을 때는 어떨까요? 온도를 모르는 동안을 정상으로 볼 수는 없습니다. 품질이 나쁘거나 값이 빠지면 유지 시간을 처음부터 다시 셉니다. 이미 발생한 경보도 데이터가 없다는 이유만으로 해제하지 않아요. 회복을 확인한 것과 관측을 못 한 것은 다르기 때문입니다.

[SLIDE 30] 이상 이벤트: 경보 한 건의 실제 모습

조건이 성립하면 감지 서비스가 경보를 남깁니다. 이상 이벤트는 이상 조건이 발생하거나 해제됐다는 기록이에요. 화면은 그중 쿨러 성능 저하가 발생한 경보입니다.

이 기록에서 볼 것은 세 가지입니다. 어느 경보인지, 무슨 일이 일어났는지, 무엇을 근거로 알렸는지예요.

경보 번호는 이 한 건을 다시 찾기 위한 식별자입니다. 번호 자체를 외우는 게 아니라, 다른 경보와 구별해 연결하는 데 쓴다고 이해하면 됩니다.

패턴은 쿨러 성능 저하입니다. 앞에서 본 복합 조건이 성립했다는 뜻이지, 원인이 핀 오염으로 밝혀졌다는 뜻은 아니에요.

근거 값에는 경보가 났을 때의 유온과 냉각 효율이 함께 담겨 있습니다. 계속 바뀌는 현재 값 대신, 그때 판정에 쓴 값을 함께 남긴 거예요. 그 밖에 설비, 심각도, 발생 상태와 시각도 이 기록에 있습니다.

잠시 뒤 유온이 내려간 상태에서 경보를 다시 열어 본다고 생각해 보세요. 현재 값만 보면 왜 경보를 냈는지 납득하기 어려울 수 있습니다. 반대로 지금 더 뜨거워졌다면 경보 당시에도 그만큼 위험했다고 잘못 생각할 수 있어요. 경보에 붙은 값은 이런 혼동을 막습니다. 지금의 상태를 알려 주는 값과 과거 판정을 설명하는 값은 역할이 달라요.

[SLIDE 31] Layer 5 · 시계열 저장소: 시각으로 찾는다

경보가 나기 전부터 온도가 오르고 있었는지 확인하려 합니다. 경보 한 건에 붙은 값으로는 변화의 모양을 볼 수 없죠. 그 시각 앞뒤의 기록을 찾아야 합니다. 이 일을 맡는 곳이 시계열 저장소입니다.

시계열 데이터베이스는 시각을 기준으로 데이터를 쌓고 시간 범위로 조회하는 저장소예요. 화면의 표에서는 계측값 하나가 한 행입니다. 시각, 설비, 태그 이름, 값이 함께 있습니다.

가운데 행은 경보가 난 때의 유온입니다. 위아래 행을 함께 읽으면 온도가 어떻게 바뀌었는지 알 수 있어요. 냉각 효율도 같은 시각의 다른 행에 저장됩니다. 센서가 늘어날 때마다 열을 늘리는 대신, 같은 형식의 행을 쌓습니다.

진동 평균을 구하려면 해당 설비의 진동 태그와 시간 구간을 골라 읽습니다. 다른 설비의 값이 섞이면 계산이 맞아도 이 경보의 근거는 되지 못하겠죠. 그래서 시각과 설비, 태그가 함께 있어야 합니다.

이벤트 로그는 읽던 위치에서 이어 읽거나 다시 처리하는 데 쓰였습니다. 시계열 저장소에서는 어느 설비의 어느 시간 구간을 볼지 정해서 값을 찾습니다. 질문하는 방식이 다른 거예요. 로그의 짧은 보존 기간이 지난 뒤에도 정해진 보존 정책 안에서 이력을 조회할 수 있습니다. 이런 저장소에는 TimescaleDB가 있습니다.

[SLIDE 32] Layer 6 · 운전 화면과 분석 화면

시계열 저장소에서 찾은 이력을 보려면 화면이 필요합니다. 지금 운전 중인 상태를 보고 조작할 화면도 있어야 하죠. 이 두 역할을 맡는 계층이 레이어 6, 관제입니다. 현재 설비를 조작하는 일과 지나간 이력을 분석하는 일을 왜 나누어 놓았는지, 두 화면의 연결을 비교해 볼게요.

왼쪽 운전 화면은 SCADA에서 사람이 보고 조작하는 화면이에요. 이런 사람과 기계 사이의 접점을 HMI라고 합니다. 운전 화면은 OT에서 브로커를 직접 구독해서 실시간 값과 알람을 보여 줘요. 운전원은 여기서 팬 속도와 펌프 부하의 설정값을 바꿉니다. 그리고 IT 쪽 서버가 멈춰도 계속 동작해요. 이 역할에는 FUXA 같은 것이 있습니다.

오른쪽 분석 화면은 이력 데이터를 그래프로 보여 주는 대시보드예요. IT에서 시계열 저장소를 조회해서 이력과 추세를 보여 줍니다. 읽기 전용이라 설비를 조작하지 않아요. 추세선 위에 경보와 조치 시점을 주석으로 표시할 수도 있습니다. 이 역할에는 Grafana 같은 것이 있어요.

왜 화면을 둘로 나눴을까요. 지금 설비를 조작해야 하는 화면은 IT에 문제가 생겨도 살아 있어야 하니까 OT에 두고, 지나간 이력을 분석하는 화면은 설비에 손댈 일이 없으니 IT에 둔 거예요. 앞에서 본 망 분리의 원칙이 화면에도 그대로 적용됩니다.

[SLIDE 33] 경보·알람·인터록의 기준과 역할

같은 유온을 보더라도 감지 서비스, 운전 화면, PLC가 하는 일은 다릅니다. 세 칸의 온도 숫자보다 그 아래의 역할부터 보겠습니다.

첫 칸은 감지 서비스의 경보예요. 유온이 55도를 넘는 것에 더해 냉각 효율과 상승 추세를 함께 확인합니다. 조건이 이어지면 원인을 진단하고 조치를 검토하기 시작합니다.

가운데는 운전 화면의 알람입니다. 유온 하나를 60도 기준과 비교해 운전원에게 높은 온도를 알립니다. 이렇게 판정을 가르는 경계값을 임계치라고 합니다.

마지막은 PLC의 인터록입니다. 유온이 65도를 넘으면 알리는 데서 그치지 않고 설비를 보호 정지시킵니다. 자동으로 풀리지 않으며, 온도가 리셋 조건 아래로 내려온 뒤 사람이 리셋해야 합니다. 화면에 적힌 리셋 조건은 55도 미만이에요.

경보와 알람은 상황을 알리고, 인터록은 정지 동작을 합니다. 먼저 난 경보를 보고 적절한 조치를 하면 보호 정지까지 가기 전에 대응할 수 있습니다. 그 대응에는 원인을 판단하는 일과 조치를 승인해 실행하는 일이 모두 필요합니다.

[SLIDE 34] Layer 7~9 · 지식·추천·승인·실행

경보를 받은 뒤 원인을 가리고 조치를 실행하기까지, 위쪽 세 계층이 일을 나누어 맡습니다.

레이어 7은 온톨로지와 원인분석입니다. 증상, 고장 유형, 원인, 조치의 관계와 판별 기준을 지식으로 보관해요. 어떤 기능을 잃었는지를 나타내는 것이 고장 유형입니다. 쿨러 사건에서는 냉각 성능 상실이죠. 이 고장 유형에서 원인 후보와 증거로 이어지는 관계를 찾습니다. 저장 도구로는 그래프 데이터베이스를 쓰며, 이 역할에는 Neo4j 같은 것이 있어요.

레이어 8은 AI 에이전트예요. 지식과 데이터를 조회해서 원인을 진단하고 추천안을 작성합니다. 쿨러 사건에서는 쿨러 핀 오염을 진단하고, 팬 최대 운전을 추천했어요. 중요한 건, 에이전트에게는 설비 명령을 보낼 권한도 수단도 없다는 겁니다.

레이어 9는 승인과 프로세스예요. 사람의 판단을 거쳐 실행하는 구조를 HITL이라고 합니다. 담당자가 추천안과 근거를 보고 승인하거나 거절해요. 그러면 프로세스 엔진이 승인된 조치만 명령으로 만들고, 결과를 확인합니다.

팬 속도나 부하를 바꾸는 일은 설비 상태와 생산에 영향을 줍니다. 그래서 에이전트는 추천안과 근거를 내고, 담당자가 검토해 승인한 뒤에만 명령을 만듭니다.

[SLIDE 35] 승인된 명령이 PLC에 닿는 경로

승인된 명령은 어떤 길로 설비까지 내려갈까요. 명령 관문과 PLC의 검사를 모두 거칩니다. 이렇게 여러 겹의 보호를 두는 방식을 심층 방어라고 해요.

왼쪽부터 따라가 볼게요. IT에 있는 프로세스 엔진이 승인된 조치를 명령으로 만들어요. 그 명령은 이벤트 로그의 명령 토픽에 기록됩니다.

DMZ에 있는 명령 관문이 그 기록을 읽어서, 명령 내용과 승인 기록을 검사해요. 통과한 명령만 OT로 넘어갑니다.

OT에서는 브로커가 그 명령을 자동 명령 토픽으로 전달하고, 엣지 게이트웨이가 받아서 PLC에 넘겨요.
마지막으로 PLC가 설비 상태를 다시 검사한 뒤에 설정값을 적용합니다.

명령서 자체가 정상이더라도, 지금 설비는 그 명령을 받을 수 없는 상태일 수 있어요. 관문은 명령 내용과 승인 기록을 검사하고, PLC는 현재 설비 상태와 설정값 범위를 다시 봅니다. 두 곳은 일부 항목을 거듭 확인하면서도 서로 다른 위험을 막아요.

[SLIDE 36] 명령 관문과 PLC가 검사하는 것

두 검사 지점에서 무엇을 확인하는지 보겠습니다. 명령이 늦게 오거나 내용이 바뀌었다면, 승인을 받았어도 그대로 실행하면 안 되겠죠. 왼쪽 명령 관문은 이런 문제를 거르고, 오른쪽 PLC는 설비가 지금 받을 수 있는지 다시 봅니다.

관문은 먼저 명령의 필수 항목과 형식을 확인합니다. 스키마 검사예요. 허용 목록에 있는 조치인지도 봅니다. 임의의 동작을 명령에 적어 보냈다고 통과시키지 않습니다.

만료 시각과 중복 여부도 확인합니다. 늦게 도착한 명령은 이미 달라진 상황에 맞지 않을 수 있고, 같은 명령이 다시 온 것은 새 요청과 구별해야 하니까요.

승인 검사에서는 승인자 이름만 읽지 않습니다. 따로 남긴 승인 기록의 내용과 실제 명령이 일치하는지 대조해요. 운전 모드가 원격 자동인지 확인하고, 너무 많은 명령이 한꺼번에 들어오지 않도록 빈도도 제한합니다.

PLC도 모드와 출처, 만료와 중복을 다시 봅니다. 명령이 관문을 지난 뒤에도 시간이 흐르고 설비 상태가 바뀔 수 있으니까요. 팬 속도와 부하가 허용 범위인지, 보호 정지 중은 아닌지도 확인합니다. 트립 중에는 일반 설정 변경을 받아들이지 않습니다.

앞에서 명령은 최소 한 번 전달하는 방식이라 중복될 수 있다고 했죠. 여기서 최근 명령 번호와 비교해 다시 온 명령을 걸러 냅니다. 화면의 관문과 PLC는 기억하는 최근 번호의 범위가 다릅니다. 핵심은 명령 번호를 붙이는 것만으로 끝나지 않고, 받는 쪽이 실제로 대조해야 한다는 점이에요.

[SLIDE 37] 명령·확인 응답·재관측의 구별

검사를 통과한 명령이 PLC에 전달됐습니다. 팬을 돌리라는 요청을 보낸 뒤에는 어디까지 확인해야 일을 마쳤다고 할 수 있을까요? 화면의 명령, 확인 응답, 재관측을 차례로 보겠습니다.

맨 위 명령에는 팬 속도를 바꾸는 내용과 승인자, 유효한 시간이 담겨 있습니다. 명령 번호는 이 요청을 다른 요청과 구별합니다.

가운데가 PLC의 확인 응답, ACK입니다. 어떤 명령을 어떻게 처리했는지 알려 주는 기록이에요. 이 사건에서는 실행을 완료했고 인터록도 정상이라고 답했습니다. 응답은 운전 상태 메시지에 실려 수집 경로로 돌아옵니다. 프로세스 엔진은 자신이 보낸 명령의 응답인지 번호를 맞춰 봅니다.

맨 아래는 그 뒤의 설비를 다시 관측한 결과입니다. 유온이 내려갔고 경보가 해제됐어요. 명령을 처리했다는 응답과, 원하는 냉각 효과가 났다는 확인을 각각 얻은 겁니다.

팬 명령에는 완료 응답이 왔는데 유온이 계속 오른다고 생각해 보세요. 명령 처리는 끝났지만 문제는 풀리지 않았습니다. 반대로 온도만 내려간 것을 보고 명령이 실행됐다고 단정할 수도 없어요. 명령 기록과 설비 상태를 연결해서 확인해야 합니다. 여기서는 유온이 회복 기준 안에 들어왔는지, 경보가 해제됐는지를 함께 봅니다.

[SLIDE 38] 데이터 리니지: 원천 데이터에서 판단 결과까지

실행 결과를 확인한 뒤에는 조치를 결정한 근거도 되짚을 수 있어야 합니다. 판단에 쓰인 값은 어디서 왔고, 어떤 계산을 거쳤을까요? 결과에서 원래 데이터까지 되짚을 수 있도록 남긴 내력이 데이터 리니지입니다.

화면의 맨 위는 센서와 PLC에서 나온 유온 기록이에요. 이것이 원천 데이터입니다. 그 아래는 그 변화에서 계산한 기울기입니다. 기울기가 양수라는 사실이 유온 상승 조건에 쓰였어요. 원천 값을 계산해서 얻은 가공 데이터입니다.

그 조건들과 냉각 효율을 확인해 경보가 남았습니다. 경보가 이상 이벤트이고, 이 경보에서 원인 진단과 조치 추천으로 이어진 것이 판단 결과예요.

이제 거꾸로 읽어 보세요. 판단에서 경보를 찾고, 경보에 쓰인 가공값과 원천 계측값으로 돌아갈 수 있습니다. 이렇게 결과에서 근거를 되짚는 성질이 추적성입니다.

이렇게 거슬러 읽을 때는 저장된 항목이 어떤 변환을 거친 값인지도 확인해야 합니다. 화면의 가공 레코드에는 그 순간의 유온과 계산된 기울기가 함께 있어요. 이름에 평균을 뜻하는 말이 보이더라도 이 레코드의 유온은 순간값입니다. 또 원천 계측값 표에는 메시지의 품질 항목을 따로 보관하지 않습니다. 어떤 변환을 거쳐 무엇이 남았는지 알아야, 기록에 없는 정보까지 있었다고 오해하지 않을 수 있어요.

[SLIDE 39] 식별자로 이어지는 판단과 작업 기록

판단 근거를 되짚으려면 서로 다른 기록이 같은 사건의 것인지도 알아야 합니다. 팬을 돌린 기록과 핀 세척 작업지시는 서로 다르지만, 같은 쿨러 사건에서 나온 일이죠. 이 기록들을 식별자로 연결하면 어떤 판단에서 나온 일인지 거슬러 볼 수 있습니다.

먼저 경보는 이상 조건이 성립했다는 기록입니다. 사건은 그 경보를 처리하는 업무 한 건이에요. 경보에는 당시 값이, 사건에는 처리가 어디까지 진행됐는지가 남습니다.

판단 기록에는 원인 진단과 조치 추천이 담깁니다. 담당자가 승인하면 팬 속도를 바꾸는 명령이 나가고, 남은 오염을 없애기 위한 작업지시도 생깁니다. 화면의 긴 번호는 이 서로 다른 기록을 구별하고 연결하기 위한 것이에요. 번호의 철자보다 무엇과 무엇이 이어지는지를 보세요.

정비 담당자가 핀 세척 작업지시를 받았다고 생각해 보겠습니다. 작업 이름만 있으면 왜 이 작업이 필요한지 다시 물어봐야 하겠죠. 판단 기록이 연결돼 있으면 원인과 근거를 확인할 수 있습니다. 판단 기록에서 경보와 당시 계측값으로도 돌아갈 수 있고요.

반대로 팬 속도 변경만 끝났다고 핀 세척까지 끝난 것은 아닙니다. 명령의 실행 상태와 정비 작업의 진행 상태를 별도로 남겨야 해요. 같은 사건에 속한 기록으로 연결해 두되, 명령 완료와 정비 완료는 따로 확인하려고 기록을 나누는 겁니다.

[SLIDE 40] 지나온 길: 아홉 기능 계층과 세 보안 영역

한 사건을 따라본 구조를 이제 전체 지도로 모아 보겠습니다. 세로로 놓인 아홉 줄은 맡은 일에 따른 계층이고, 오른쪽 세 칸은 구성 요소가 놓인 보안 영역입니다.

아래쪽은 값을 만들고 전달하는 부분입니다. 설비와 제어기, 엣지와 브로커, 이벤트 로그가 있어요. 가운데는 흐르는 값을 감지하고 이력을 저장해 사람에게 보여 주는 부분입니다. 위쪽은 지식을 조회해 추천하고, 사람이 승인한 일을 실행하는 부분이에요.

오른쪽 점은 각 역할이 어느 망에 놓이는지 보여 줍니다. 관제 계층을 보세요. 운전 화면은 OT에 있고, 이력을 읽는 분석 화면은 IT에 있습니다. 같은 관제라도 필요한 연결이 달라 서로 다른 영역에 놓입니다.

따라서 계층과 보안 영역을 같은 구분으로 외우면 안 돼요. 계층은 무엇을 맡는지, 보안 영역은 어디와 연결되는지를 보여 줍니다.

[SLIDE 41] 데이터 수집과 승인된 자동 명령의 두 경로

아홉 계층을 지나는 데이터와 명령을 두 경로로 연결해 보겠습니다. 데이터는 OT에서 IT로 수집하고, 승인된 자동 명령은 별도 경로의 검사를 거쳐 설비에 적용해요.

위쪽 실선이 올라가는 길입니다. PLC에서 나온 유온 값이 엣지 게이트웨이와 MQTT 브로커를 거쳐요. DMZ의 수집 서비스가 그 값을 IT의 이벤트 로그로 옮기고, 감지 서비스가 그 기록을 읽어 이상을 판단합니다.

아래쪽 점선이 내려가는 길이에요. IT의 프로세스 엔진에서 승인된 팬 속도 변경 명령이 출발합니다. 이벤트 로그의 명령 토픽에 기록되고, DMZ의 명령 관문이 검사하고, 브로커와 엣지를 거쳐 PLC의 팬 속도 설정값에 닿아요.

브로커와 엣지처럼 두 방향에서 함께 쓰는 구성요소도 있지만, 수집과 자동 명령의 역할은 나뉩니다. 수집 서비스는 OT로 발행하지 않고, 자동 명령은 별도 관문과 PLC의 검사를 거쳐야 해요. 처음에 던진 물음, 숫자 하나가 어떻게 판단과 조치로 이어지는가에 대한 답이 이 두 경로에 담겨 있습니다.

[SLIDE 42~43] 실습: 실행 환경 구성과 서비스 동작 확인

시스템 구조를 직접 확인하기 위한 실행 환경 실습을 시작하겠습니다. 데이터 수집과 명령 전달을 맡는 서비스를 준비된 환경에서 실행할 거예요. 목표는 컨테이너 상태와 주요 서비스의 접속을 확인하고 포털이 열리는 상태를 만드는 것입니다.

실습에 쓸 도구의 역할부터 짚어 둘게요. VS Code는 파일과 터미널을 다루는 작업 공간입니다. GitHub Copilot에는 자연어로 요청해 파일을 읽고 고치는 일을 맡길 수 있어요. AI에게 요청하고 코드를 실행하며 고쳐 가는 방식을 바이브 코딩이라고 합니다. 서비스와 랩 코드는 Python으로 작성돼 있습니다.

Docker Desktop은 컨테이너를 실행하고, Docker Compose는 한 파일에 정의된 여러 서비스를 함께 실행합니다. 실행한 뒤에는 상태와 접속 결과를 정상 구성과 대조합니다. 마지막에는 완성 환경에서 어떤 서비스가 연결돼 있고 포털에서 무엇을 확인할 수 있는지 설명하고 다시 실행해 보겠습니다.

[SLIDE 44~45] 실습: 설비 이상에서 조치까지 전체 과정 시연

쿨러 사건의 처리 과정을 확인하는 실습입니다. 실행 환경 실습에서 열어 둔 포털에서 경보와 원인 판별에 필요한 값을 확인하고, 조치가 추천되는 과정을 따라갑니다. 담당자가 승인하면 명령이 나가고, PLC의 응답을 받은

뒤에도 값을 다시 관측해서 상태가 완화됐는지 확인해요. 유온이 내려가도 핀 오염은 남아 있으므로 세척 작업지시는 따로 남습니다.

그런데 나중에 작업지시만 받았다면, 왜 핀을 세척하라고 했는지 어디서 확인할 수 있을까요? 이번에는 기록을 거꾸로 따라가 보겠습니다. 작업지시가 참조하는 판단 기록을 찾고, 그 판단이 가리키는 사건을 거쳐 처음 경보로 돌아갑니다.

남길 것은 근거 연결표입니다. 기록 이름만 순서대로 적지 않고, 다음 기록을 찾는 데 쓴 참조 칸도 함께 표시하세요. 경보가 남긴 관측값과 원인을 가릴 때 쓴 증거도 구별해 봅니다. 결과 검토에서는 각 연결의 참조가 맞는지 확인하고, 마지막에는 강사가 완료 기록을 열어 완성 연결표와 대조하겠습니다.

[SLIDE 46] 수치만으로 조치를 정할 수 없는 이유

세 번째 단원은 수치만으로 조치를 정할 수 없는 이유를 다룹니다. 쿨러 사건의 근거 연결표에는 경보와 조치 사이에 원인 판단이 있었죠. 유온이 올랐다는 사실을 알고도 바로 팬 속도를 올리지 않은 까닭은 무엇일까요? 원인을 가리고 조치를 고르는 데 필요한 지식을 이 사건에서 찾아보겠습니다.

[SLIDE 47] 경보 이후의 판단과 승인, 조치 뒤의 변화

쿨러 사건의 그래프에서 경보와 조치 사이를 먼저 보겠습니다. 위쪽은 유온, 아래쪽은 냉각 효율이에요. 쿨러 성능이 떨어진 뒤 기름 온도가 오르고 경보가 났습니다. 경보에서 승인까지 색칠한 구간을 보세요. 그동안에도 온도는 계속 올랐습니다.

이 구간은 원인을 가리고 조치안을 비교한 뒤 사람의 승인을 기다린 시간입니다. 기록에서는 약 1분 48초가 걸렸어요. 화면에는 진단·추천과 승인 대기에 걸린 시간도 따로 적혀 있습니다.

승인 지점에서 팬 최대 운전을 선택했고, 곧 명령이 나갔습니다. 그 뒤 위쪽 유온 곡선이 내려갑니다. 마지막으로 표시된 지점에서는 경보가 해제됐어요. 그런데 아래쪽 냉각 효율은 이전보다 올라왔어도 기준선 아래에 남아 있습니다.

체온계가 열이 난다는 사실은 알려 줘도 병의 원인까지 알려 주지는 않죠. 경보도 조건이 성립했다는 것만 알려 주므로, 원인과 조치까지 가려면 지식이 필요합니다.

이 그래프에서 풀어야 할 물음은 두 가지입니다. 경보 뒤에 무엇을 확인하느라 시간이 필요했는지, 온도가 내려갔는데 왜 냉각 효율은 회복되지 않았는지예요.

[SLIDE 48] 데이터, 정보, 지식과 판단의 네 질문

표의 냉각 효율 값에서 출발해, 데이터와 정보, 지식이 어떻게 다른지 짚어 보겠습니다. 표의 첫 줄에는 숫자만 있습니다. 온도인지 압력인지조차 알 수 없죠. 그 아래에 설비 이름과 냉각 효율이라는 대상, 퍼센트라는 단위가 붙으면 의미를 읽을 수 있습니다.

데이터는 관측하거나 기록한 값입니다. 여기에 대상과 단위, 기준 같은 맥락을 붙이면 정보가 됩니다. 지금 냉각 효율이 얼마인지, 정상으로 보는 기준보다 낮은지까지 답할 수 있어요.

그런데 낮다는 것을 안 다음에는 어떻게 할까요? 표의 아래 두 줄에서 지식이 필요해집니다. 냉각 효율은 낮고 팬 진동은 정상이라는 정보를 함께 봐서 핀 오염을 판단합니다. 또 핀 오염이면 세척한다는 연결로 조치를 찾습니다.

이렇게 정보를 원인으로, 원인을 조치로 이어 판단에 쓸 수 있게 한 것이 여기서 말하는 지식입니다. 같은 수치를 가지고도 어떤 정보를 함께 볼지, 그 조합을 어떻게 해석할지 알아야 원인을 가릴 수 있어요.

표의 오른쪽 질문을 따라가면 차이가 보입니다. 얼마인지와 기준을 벗어났는지는 값과 정보로 답합니다. 왜 그런지와 무엇을 할지는 그 정보를 연결하는 지식으로 답합니다. 그 지식이 실제로 어떤 내용인지 원인 후보 표에서 확인하겠습니다.

[SLIDE 49] 원인 후보와 증거의 대조

쿨러가 기름을 충분히 식히지 못하는 까닭을 세 후보로 나누었습니다. 핀이 먼지나 유막으로 막혔을 수도 있고, 팬이 제 속도를 못 낼 수도 있고, 주변 공기가 더울 수도 있어요. 유온 상승만 보면 셋이 비슷하게 보입니다.

그래서 후보마다 확인할 값을 정합니다. 화면 가운데에 있는 것이 증거입니다. 여기에는 무엇을 어느 구간에서 읽고, 어떤 기준과 비교할지가 들어 있어요.

첫 행은 핀 오염입니다. 냉각 효율은 낮지만 팬 진동은 정상이어야 합니다. 실제 평균값을 대조하니 두 조건이 모두 맞아요. 팬이 문제가 아니라면 냉각을 막는 다른 자리를 볼 이유가 생기는 거죠.

두 번째 후보는 팬 구동 불량입니다. 같은 팬 진동 평균을 이번에는 이상 기준과 대조합니다. 실제 값은 그 기준보다 낮으므로 이 후보의 조건에는 해당하지 않아요.

마지막은 주변 온도 상승입니다. 캐비닛 온도 평균이 기준 이상인지 봅니다. 여기서도 실제 값은 기준보다 낮아요. 그래서 이 사건에서는 핀 오염을 원인으로 판단했습니다.

팬 진동 하나가 두 후보를 가른다는 점을 보세요. 낮은 진동은 핀 오염의 조건에는 맞고, 팬 구동 불량의 조건에는 맞지 않습니다. 하나의 값이 어떤 후보를 확인하는지에 따라 쓰임이 달라져요.

또 지금 화면에 뜬 순간값을 그대로 비교하지 않았습니다. 표에 적힌 구간의 평균을 썼어요. 확인할 대상뿐 아니라 읽는 구간과 기준까지 알아야 같은 판단을 다시 해 볼 수 있습니다. 원인 이름을 외우는 대신, 가운데 칸의 근거에서 오른쪽 판정이 나오는지 따라 읽어 보세요.

[SLIDE 50] 원인에 따라 달라지는 근본 조치

원인을 가르면 손볼 곳이 달라집니다. 핀 오염이면 쿨러 핀을 세척합니다. 막힘을 없애 열이 빠져나가게 하는 거예요. 팬 구동 불량이면 팬의 벨트를 점검합니다. 팬이 제 속도로 돌아 바람을 보낼 수 있어야 하니까요. 주변 공기가 더우면 캐비닛 환기를 보강합니다.

세 조치는 모두 냉각에 관계있지만 같은 작업은 아닙니다. 핀이 막혀 있는데 벨트만 점검하면 핀에 낀 오염은 남습니다. 환기를 보강해도 막힌 핀 자체를 씻어 내지는 못하죠.

그래서 근본 조치를 고를 때는 원인과 연결해야 합니다. 온도가 올랐다는 관측에서 바로 작업 이름으로 뛰어넘지 않고, 어느 자리에서 무엇이 잘못됐는지를 확인하는 이유입니다.

[SLIDE 51] 완화 조치와 근본 조치

그런데 실제 사건에서 먼저 승인된 것은 핀 세척이 아니라 팬 최대 운전이었습니다. 핀 오염이 원인이라면서 왜 팬부터 돌렸을까요?

왼쪽은 완화 조치입니다. 팬을 더 돌려 당장 유온을 낮춥니다. 하지만 오염은 그대로여서 냉각 효율은 기준 아래에 머물렀어요. 그래프에서 온도는 내려왔지만 냉각 효율이 회복되지 않은 이유입니다.

오른쪽은 근본 조치입니다. 막힌 핀을 세척해 원인을 없앱니다. 이 작업은 설비를 멈추고 해야 하므로 정지할 시간도 잡아야 해요. 그래서 별도의 작업지시로 남겼습니다.

완화 조치는 정비할 여유를 벌어 주지만 계속 미룰 수는 없습니다. 매뉴얼은 팬 최대 운전을 연속 24시간 이내로 제한합니다. 이보다 오래 돌리면 팬 수명에 영향을 주기 때문이에요.

승인 책임도 구별합니다. 운전원은 팬 최대 운전을 승인했고, 설비를 세우는 핀 세척은 작업지시를 통해 설비보전팀장이 승인합니다. 같은 사건에서도 급한 증상을 낮추는 일과 원인을 없애는 일을 따로 관리합니다.

[SLIDE 52] 같은 조치가 원인에 따라 설비를 세우는 경우

그럼 원인을 찾는 동안 우선 팬을 최대로 돌리면 어떨까요? 팬 자체에 문제가 있는 경우에는 오히려 위험할 수 있습니다.

왼쪽의 핀 오염 설비에서는 팬 속도를 올리자 바람이 늘고 유온이 내려갔습니다. 오른쪽은 팬 베어링이 닳은 설비예요. 같은 속도로 올렸는데 진동이 보호 기준을 넘어서 PLC가 설비를 세웁니다.

베어링이 닳은 팬에서는 속도를 올릴수록 마모로 인한 진동이 크게 늘어납니다. 냉각에는 도움이 되는 동작이라도 다른 위험을 키울 수 있어요. 화면의 온도 하락과 진동 상승은 서로 다른 결과를 보여 줍니다.

조치를 고르는 근거는 원인 판단이고, 실행할 때 설비를 보호하는 일은 PLC가 맡습니다. 원인 판단과 현장의 보호가 함께 있어야 냉각 효과만 보고 위험한 조치를 내리는 일을 막을 수 있어요. 따라서 같은 경보가 났어도 원인을 가린 뒤에 완화 조치를 골라야 합니다. 경보 수치만으로 조치를 정할 수 없는 이유예요.

[SLIDE 53] 판단 근거의 연결

네 번째 단원의 주제는 판단 근거의 연결입니다. 원인을 가릴 때 썼던 계측값과 매뉴얼의 판별 기준은 한곳에 모여 있지 않습니다. 자료가 따로 있어도 하나의 판단에 쓸 수 있으려면, 그 사이에 무엇을 연결해야 할까요?

[SLIDE 54] 판단에 쓰이는 정보가 놓인 곳

쿨러 경보 주변의 네 곳이 판단에 필요한 정보의 출처입니다. 그림의 점선은 이 출처들 사이의 연결이 아직 정리되지 않았다는 뜻이에요. 실제 사건에서 쓴 정보를 출처별로 나누어 놓고, 무엇을 어디서 가져왔는지 짚어 볼게요.

왼쪽 위 시계열 저장소에는 냉각 효율과 캐비닛 온도 같은 관측값이 있습니다. 왼쪽 아래 운전 상태에서는 원격 자동 모드인지, 팬이 얼마나 도는지 확인합니다. 설비가 움직이는 동안 바뀌는 정보들이에요.

오른쪽 위 매뉴얼에는 원인별로 어떤 값을 비교하라는 기준이 있습니다. 오른쪽 아래 MES는 생산 지시와 실적, 납기를 관리하는 제조 실행 시스템입니다. 여기서 긴급 오더의 납기까지 약 여섯 시간이 남았다는 정보를 읽었습니다.

납기는 왜 필요할까요? 핀 세척은 설비를 멈춰야 하는 작업입니다. 원인을 없애는 조치라도 지금 멈췄을 때 생산에 어떤 영향이 있는지 함께 봐야 합니다. 원인을 가리는 근거와 조치 시점을 고르는 근거가 서로 다른 곳에 있는 거예요.

매뉴얼의 기준이 어느 저장소의 어떤 값과 이어지는지까지 알아야, 따로 있는 자료를 하나의 판단에 쓸 수 있습니다.

[SLIDE 55] 매뉴얼의 문장에 들어 있는 것과 없는 것

판별 기준의 출처인 매뉴얼을 펼쳐 보겠습니다. 냉각 효율 평균이 낮고 팬 진동 평균은 정상이면 핀 오염으로 판정한다는 문장입니다. 원인 후보와 확인할 값, 기준이 들어 있죠.

매뉴얼을 읽을 때는 '가장 흔하다'와 '지금 원인이다'를 구별해야 합니다. 흔한 원인이라는 정보는 후보를 검토하는 데 참고할 수 있지만, 지금 설비에서는 실제 값을 판별 기준에 대어 봐야 합니다. 쿨러 사건에서 핀 오염을 판단한 근거는 냉각 효율과 팬 진동의 조건이 함께 맞았다는 것이었죠.

사람은 매뉴얼의 판별 문장을 보고 운전 화면이나 기록에서 값을 찾아볼 수 있습니다. 컴퓨터가 같은 일을 하려면 연결을 더 명확히 적어야 해요.

먼저 지금 경보가 어느 매뉴얼 대목에 해당하는지 찾아야 합니다. 다음으로 그 대목에서 말하는 값을 어느 태그와 시간 구간으로 읽을지 정해야 합니다. 이 문장에는 팬 진동 태그는 있지만 냉각 효율의 태그 이름과 저장소 조회 방법까지 있지는 않습니다.

핀 오염이라고 판단한 뒤에는 어떤 조치가 맞는지도 이어야 합니다. 완화 조치와 근본 조치는 같은 매뉴얼의 다른 절에 있어요. 이 판별 문장만 읽어서는 실행할 조치까지 정해지지 않습니다.

문서의 문장을 찾는 데서 그치지 않고, 그 문장이 요구하는 확인과 조치로 이어지는 연결까지 갖춰야 합니다.

[SLIDE 56] 숙련자의 판단 경로와 기록되지 않은 연결

숙련자는 매뉴얼과 계측값처럼 흩어진 정보를 머릿속에서 연결합니다. 유온 상승을 보고 냉각 기능에 문제가 생겼다고 짐작하고, 가능한 원인을 떠올린 뒤 확인할 센서를 찾아갑니다.

표의 가운데 칸을 따라가 볼게요. 첫 행은 증상에서 고장 유형과 원인 후보로 가는 연결입니다. 둘째는 후보마다 확인할 값을 찾아 기준과 비교하는 연결이에요. 셋째는 원인에 맞는 조치를 고르고 생산 일정까지 고려하는 과정입니다.

이 연결을 숙련자만 알고 있으면, 그 사람이 없을 때 판단이 늦어질 수 있습니다. 같은 정보를 보고도 후보를 떠올리는 순서가 달라질 수 있고요. 연결을 기록하면 다른 사람도 어느 증거를 확인했는지 검토할 수 있습니다. 컴퓨터가 같은 경로를 따라갈 수도 있어요.

여기서 기록할 것은 경험에서 나온 결론 하나가 아닙니다. 어떤 증상에서 어느 후보로 갔고, 무엇을 확인해 조치를 골랐는지입니다. 가운데 증거 확인을 빠뜨렸다면 그 빠진 부분도 드러나야 합니다. 판단의 경로를 밖으로 꺼내 쓰는 이유예요.

[SLIDE 57] 표·문서·코드에서 지식을 분리하는 이유

숙련자가 쓰던 판별 기준을 프로그램의 조건문에 옮기면 컴퓨터도 그 기준으로 판단할 수 있습니다. 그런데 매뉴얼이 바뀌면, 그 기준이 들어간 프로그램도 찾아 고쳐야 하죠. 기준과 프로그램을 별개로 관리할 방법이 필요합니다.

왼쪽 위는 지식을 표로 펼친 예입니다. 한 고장 유형에 여러 원인과 증거가 붙으니 고장 유형이 반복됩니다. 각 행을 읽을 수는 있지만, 다른 표의 대상과 어떻게 이어지는지도 따로 정해야 합니다.

문서에는 판별 기준이 문장으로 담겨 있어요. 사람은 해석할 수 있지만 같은 대상의 이름이 문서마다 달라지기도 합니다. 아래쪽 코드 예는 그 기준을 조건문으로 옮긴 것입니다. 조건이 맞으면 핀 오염이라는 결과를 냅니다.

오른쪽 마지막 칸은 대상과 관계, 판별 기준을 프로그램 밖의 지식으로 두는 방식입니다. 사람은 지식과 출처를 검토하고, 프로그램은 그 지식을 읽어 판정합니다. 기준을 바꾸는 일과 판단 프로그램을 고치는 일을 나눌 수 있어요.

표나 코드가 나빠서 쓰지 말자는 이야기는 아닙니다. 판단에 쓰는 의미와 출처가 여러 곳에 흩어지지 않게 하자는 것입니다. 매뉴얼이 개정되면 어느 기준을 고쳐야 하는지 찾을 수 있어야 해요. 앞에서 학습 기록에 출처를 연결한 것과 같은 이유입니다.

[SLIDE 58] AI에게 그대로 묻는 방식과 지식을 조회하게 하는 방식

AI가 원인을 판단할 때도 이 설비의 지식이 필요합니다. '유압 설비의 유온이 오르는데 원인이 뭘까'라고만 물으면 여러 설명을 내놓을 수 있습니다. 하지만 지금 설비의 매뉴얼과 값을 주지 않았다면, 그 답으로 이 설비의 원인을 확인한 것은 아니에요.

오른쪽 방식에서는 지식에 적힌 원인 후보와 판별 기준을 조회합니다. 이어서 정해진 조회 방법으로 저장소의 값을 읽어 대조해요. 어떤 기준과 값을 썼는지 기록에도 남깁니다.

그때 평균값만 보지는 않습니다. 필요한 구간에 관측값이 충분한지, 중간에 긴 공백은 없는지도 확인해요. 화면 아래의 표본 수와 가장 긴 간격은 쿨러 사건에서 이 확인을 한 기록입니다.

값이 대부분 빠졌어도 몇 개 남은 값으로 평균은 계산할 수 있겠죠. 하지만 관측하지 못한 동안 설비가 어땠는지는 알 수 없습니다. 그래서 값이 오래됐거나 관측이 부족하면 판정을 보류합니다.

이 과정에서 증거의 조회문은 AI가 즉석에서 지어내지 않고 지식에 적힌 것을 사용합니다. 같은 기준과 같은 관측값으로 결과가 맞는지 다시 확인할 수 있어요. 문서의 기준과 저장소의 값을 연결하면, AI가 어떤 근거로 판단했는지 확인할 수 있습니다.

[SLIDE 59] 온톨로지의 정의와 관계

마지막 이론 단원은 온톨로지의 정의와 관계입니다. 원인 후보와 판별 기준을 조회하려면, 기록에 쓰인 대상과 관계의 뜻부터 같아야 합니다. 같은 부품을 서로 다른 대상으로 기록하거나 관계를 다르게 해석하면 필요한 근거를 놓칠 수 있겠죠. 온톨로지가 그 뜻을 정하는 방법을 배우고, 쿨러의 관계 그림에 적용해 보겠습니다.

[SLIDE 60] 온톨로지: 공통 의미를 정의하는 지식 모델

온톨로지는 한 분야의 대상과 관계를 명시적으로 정의한 지식 모델입니다. 명시적이라는 것은 머릿속에서 알아서 해석하게 두지 않고, 함께 확인할 수 있게 적는다는 뜻입니다. 이름만 통일하는 것이 아니라 무엇을 뜻하고 어떻게 연결되는지까지 정합니다. 사람과 컴퓨터가 같은 뜻으로 쓰도록 종류, 속성, 관계, 제약을 정하는 거예요.

먼저 종류를 나눕니다. 오일 쿨러는 구성 요소이고 유온 센서는 센서입니다. 유온 상승은 증상이고 핀 오염은 원인이에요. 모두 이름이 있지만 같은 종류의 대상은 아닙니다.

속성은 대상이 갖는 값을 담는 항목입니다. 유온의 단위, 정상값, 한계가 그 예예요. 단위 칸에는 섭씨를, 정상값과 한계 칸에는 설비를 판단하는 기준값을 둡니다.

관계는 대상 사이의 뜻을 이름과 방향으로 정합니다. 냉각 성능 상실에서 팬 최대 운전으로 가는 관계는 완화된다는 뜻이에요. 핀 세척으로 가는 관계는 근본적으로 해결된다는 뜻입니다. 두 조치를 선으로만 이으면 같은 연결처럼 보이지만, 관계 이름을 붙이면 차이가 드러납니다.

제약은 올바른 연결의 조건입니다. '일으킨다'는 관계를 원인에서 고장 유형으로 잇도록 정의했다고 해 볼게요. 그 자리에 증상과 조치를 넣으면 양 끝의 종류가 맞지 않습니다. 이 조건을 기준으로 잘못된 연결을 찾아낼 수 있어요.

정의를 적는 일과 실제로 검사하는 일은 구분돼요. 제약이 검사 기준을 주고, 검증 단계가 그 기준을 적용합니다.

[SLIDE 61] 같은 대상을 가리키는 여러 말

정비 기록에는 '열교환기', 설비 목록에는 '오일 쿨러'라고 적혀 있습니다. 두 말이 같은 부품을 가리키는지 모르면 기록을 따로 찾게 되겠죠. 그래서 하나의 대상에 다른 이름을 함께 등록합니다.

화면 마지막 카드는 오일 쿨러에 쿨러와 열교환기라는 이름을 묶은 예입니다. 앞의 카드들은 증상이에요. 유온 상승에는 온도 상승과 과열이라는 표현이 연결돼 있어요. 냉각 효율 저하에는 냉각 불량, 진동 증가에는 떨림과 소음이 연결돼 있습니다.

이렇게 이름을 묶으면 서로 다르게 부르던 말을 같은 대상에 연결할 기준이 생깁니다. 다만 '소음'이라는 말이 모든 상황에서 팬 진동을 뜻하는 것은 아니겠죠. 여기서는 이 설비의 증상을 가리키는 표현으로 등록한 것입니다.
문장에서 어떤 대상을 뜻하는지는 그 맥락과 함께 확인합니다.

온톨로지의 공통 의미는 이런 일을 가능하게 합니다. 현장에서 쓰는 말을 없애는 대신, 그 말들이 어떤 대상을 가리키는지 분명히 하는 거예요.

[SLIDE 62] 온톨로지·지식 그래프·그래프 DB·관계 그림

이렇게 뜻을 정한 지식은 실제 대상으로 채우고, 저장하고, 그림으로 나타낼 수 있습니다. 이 과정에서 쓰는 네 용어를 구별해 볼게요.

온톨로지는 공통 정의입니다. 원인과 고장 유형이라는 종류가 있고, 원인은 고장 유형을 일으킬 수 있다고 정합니다. 여기에 핀 오염과 냉각 성능 상실이라는 대상을 넣어 연결하면 지식 그래프가 됩니다. 정의에 맞춰 채운 지식이에요.

그래프 DB는 그 대상과 연결을 저장하고 조회하는 도구입니다. 이런 도구에는 Neo4j가 있습니다. 저장된 연결을 원과 화살표로 나타내 사람이 보게 한 것이 관계 그림이에요.

또 계속 바뀌는 온도까지 지식 그래프에 모두 넣는 것은 아닙니다. HYD에서는 확인할 값과 읽는 방법을 지식으로 두고, 실제 관측값은 시계열 저장소에서 읽습니다.

그러니 그림을 그렸다는 것, DB에 저장했다는 것, 뜻을 정의했다는 것은 같은 일이 아닙니다. DB에 선이 잘 저장돼 있어도 관계의 뜻은 틀릴 수 있습니다. 온톨로지가 준 기준으로 그 연결을 검토해야 해요.

[SLIDE 63] 노드·관계·방향을 문장으로 읽는다

관계 그림을 읽을 때는 화살표 하나를 문장으로 바꿔 보세요. 왼쪽 원은 쿨러 핀 오염, 오른쪽 원은 쿨러 냉각 성능 상실입니다. 대상 하나를 나타내는 원이 노드이고, 둘을 잇는 이름 있는 연결이 관계입니다.

화살표에는 CAUSES라는 이름이 붙습니다. '일으킨다'는 뜻이에요. 출발 노드를 주어, 관계를 서술어, 도착 노드를 목적어로 읽으면 '핀 오염은 냉각 성능 상실을 일으킨다'가 됩니다.

화살표를 거꾸로 그렸다고 해 보세요. '냉각 성능 상실이 핀 오염을 일으킨다'가 됩니다. 선 하나가 있다는 점은 같아도 원인과 결과가 뒤집혔죠. 방향도 뜻의 일부입니다.

다른 관계도 같은 방식으로 읽습니다. '유온 상승은 냉각 성능 상실을 가리킨다.' '냉각 성능 상실은 팬 최대 운전으로 완화된다.' 관계마다 서술어가 달라집니다.

문장으로 읽기는 그림을 점검하는 방법입니다. 말이 자연스럽게 이어지는지 보고, 실제 사실과 양 끝 대상의 종류까지 맞는지 확인합니다.

[SLIDE 64] HYD 지식 그래프 지도: 설비 쪽과 진단 · 조치 쪽

화살표 하나를 문장으로 읽을 수 있으면, 여러 관계가 모인 지도에서도 뜻을 따라갈 수 있습니다. 가운데 진한 원은 고장 유형입니다. 쿨러 냉각 성능 상실을 이 자리에 놓고 주변의 연결을 따라가 볼게요.

왼쪽 아래 증상은 관측한 모습입니다. 유온 상승이라는 증상이 냉각 성능 상실을 가리킵니다. 오른쪽 위 원인은 왜 그런 일이 생겼는지 설명합니다. 핀 오염이 냉각 성능 상실을 일으켜요. 증상과 원인이 가운데 고장 유형에서 만납니다.

같은 냉각 성능 상실에도 원인은 여럿일 수 있습니다. 그래서 증상에서 곧바로 핀 오염으로 뛰어가지 않고, 고장 유형에 붙은 후보를 모아 증거로 가릅니다.

왼쪽 위 구성 요소는 고장이 어느 부품에서 일어나는지 알려 줍니다. 냉각 성능 상실은 오일 쿨러에서 발생해요. 그 바깥으로 가면 오일 쿨러를 가진 설비와 이를 계측하는 센서가 있습니다. 증상도 관측한 센서와 이어져 있으므로, 어느 설비의 어느 값을 확인할지 찾을 수 있습니다.

오른쪽 아래에는 조치 방법이 있습니다. 냉각 성능 상실은 팬 최대 운전으로 완화됩니다. 여기서 매뉴얼 절까지 연결되면 이 조치의 문서 근거를 찾을 수 있어요. 실제로는 조치 방법의 단계를 거쳐 매뉴얼 절로 가는데, 지도에서는 그 단계를 줄여 화살표 하나로 나타냈습니다.

이 지도에서 '어디가 기능을 잃었는가'와 '왜 그랬는가'를 구별해 보세요. 기능을 잃은 자리는 오일 쿨러여도 원인은 팬이나 주변 공기에 있을 수 있습니다. 고장이 난 구성 요소를 아는 것과 손볼 원인을 가리는 것은 다른 확인입니다.

설비와 센서의 연결은 어느 대상의 값을 볼지 정해 주고, 진단 쪽 연결은 그 값으로 무엇을 판단할지 정해 줍니다. 같은 정의와 판별 기준을 적용할 수 있는 설비라면, 그 설비의 실제 값을 읽어 이 진단 지식을 사용할 수 있습니다.

[SLIDE 65] 관계의 방향: 저장된 방향과 따라가는 방향

지도를 보면 의문이 하나 생겨요. 경보에서 출발하면 고장 유형에 먼저 닿는데, 「일으킨다」 화살표는 원인에서 고장 유형 쪽으로 향해 있죠. 그럼 고장 유형에서 원인을 어떻게 찾을까요. 관계 이름의 뜻이 저장 방향을 정하지만, 조회할 때는 새 관계를 만들지 않고 그 방향을 거슬러 갈 수 있습니다.

왼쪽이 저장된 방향이에요. 「일으킨다」 관계는 쿨러 핀 오염에서 쿨러 냉각 성능 상실로 저장돼 있습니다. 원인이 주어니까요. 「핀 오염은 냉각 성능 상실을 일으킨다.」 관계 이름의 뜻이 화살표의 방향을 정한 거예요.

오른쪽이 따라가는 방향입니다. 경보에서 출발한 컴퓨터는 고장 유형에 먼저 도착하죠. 고장 유형으로 들어오는 관계를 거슬러 원인 후보를 찾습니다. 이때 저장된 관계를 그대로 써요. 같은 사실을 반대 방향으로 표현하는 관계를 새로 만들 필요가 없습니다.

화살표는 한쪽으로만 다녀야 하는 일방통행 표시가 아니에요. 어느 쪽에서든 따라갈 수 있습니다. 잘못된 건 거슬러 조회하는 일이 아니라, 관계 이름의 뜻과 저장 방향이 어긋난 경우예요. 예를 들어 「일으킨다」라는 관계인데 고장 유형에서 원인 쪽으로 저장돼 있다면, 「냉각 성능 상실이 핀 오염을 일으킨다」가 되어 사실과 반대가 됩니다.

[SLIDE 66] 관계를 따라가는 과정 ①: 경보에서 고장 유형까지

관계의 방향을 알았으니, 쿨러 경보에서 출발해 지도 안의 지식을 찾아보겠습니다. 왼쪽 위 이상 패턴은 쿨러 성능 저하입니다. 경보 기록의 패턴 칸이 가리키던 정의예요.

이상 패턴에는 앞에서 본 경보 조건과 유지 시간이 정의돼 있습니다. 감지 서비스는 그 정의를 읽어 경보를 내고, 진단은 경보의 패턴 이름을 이용해 같은 정의에서 출발합니다. 경보 한 건은 어느 설비에서 언제 조건이 성립했는지 남긴 기록이고, 패턴은 그 조건의 정의예요. 서로 다른 경보도 같은 패턴을 가리킬 수 있지만, 증거 값은 각 경보의 설비에 맞춰 읽어야 합니다.

패턴에서 증상으로 가는 관계는 무엇을 감지하는지 나타냅니다. 여기서는 유온 상승입니다. 그 증상은 쿨러 냉각 성능 상실이라는 고장 유형을 가리켜요.

그림의 회색 원들은 아직 따라가지 않은 부분입니다. 지금은 출발점과 도착점만 짚으면 됩니다. 경보의 패턴에서 증상을 거쳐, 가운데 고장 유형인 쿨러 냉각 성능 상실까지 찾아왔습니다.

[SLIDE 67] 관계를 따라가는 과정 ②: 원인 후보, 증거, 조치

고장 유형에 도착했으니 원인 후보를 찾습니다. 오른쪽의 원인 노드 세 개에서 가운데로 화살표가 들어옵니다. 이 관계를 거슬러 핀 오염, 팬 구동 불량, 주변 온도 상승을 찾습니다.

원인 옆에는 증거가 연결돼 있어요. 핀 오염은 냉각 효율과 팬 진동을 확인합니다. 다른 두 후보는 팬 진동과 캐비닛 온도를 봅니다. 증거에 적힌 조회 방법으로 값을 읽고 기준에 대어 봅니다.

확인 결과를 맨 위의 파란 원으로 표시했습니다. 핀 오염의 두 조건이 맞았고, 다른 두 후보는 조건에 맞지 않았어요. 조치 방법은 아래쪽에 있습니다. 고장 유형에서 완화 조치 관계를 따라 팬 최대 운전을 찾습니다.

관계를 따라가는 일은 후보와 확인할 근거를 찾는 일입니다. 원인이 확정되는 것은 실제 값을 기준과 대조한 뒤예요. 어떤 후보도 증거로 뒷받침되지 않거나 필요한 조회가 불완전하면 조치를 보류합니다. 값을 읽지 못한 것을 '조건에 맞지 않는다'고 처리하면 안 됩니다. 확인한 결과와 확인하지 못한 상태를 구별해야 합니다.

[SLIDE 68] 판단 기록에 남기는 기준·값·조치의 근거

원인을 가려 조치를 추천한 뒤에는, 담당자가 그 근거를 검토할 수 있어야 합니다. '핀 오염이니 팬을 돌리자'는 결론만 있으면 충분할까요? 표는 그 결론을 확인할 수 있도록 함께 남긴 근거입니다.

첫 행에는 핀 오염의 두 판별 기준과 실제 평균이 있습니다. 둘째에는 다른 후보를 고르지 않은 이유가 있어요. 이미 계산한 값을 다시 읽기보다, 각 판정에 근거가 붙어 있는지 보겠습니다.

셋째는 그 값의 출처입니다. 지식 그래프는 확인할 태그와 집계 구간, 판별 기준을 주고, 시계열 저장소는 그 구간의 관측값을 줍니다. 정의와 관측을 연결해 남긴 거예요.

넷째는 조치의 문서 근거입니다. 고장 유형에서 완화 조치를 찾고, 그 조치 방법의 단계를 거쳐 매뉴얼 절에 닿습니다. 사람이 추천안을 확인할 때 왜 이 조치를 쓸 수 있는지 원문으로 돌아갈 수 있습니다.

마지막에는 비교한 다른 조치도 남습니다. 팬 최대 운전이 가장 높은 순위였고, 부하를 낮추는 두 안은 공동 2순위였습니다. 설비를 지금 멈춰 핀을 세척하는 안은 맨 아래였어요. 순위에는 냉각 효과뿐 아니라 납기 같은 업무 값도 반영됐습니다. 담당자는 고른 안과 함께 다른 대안도 검토할 수 있습니다.

[SLIDE 69] 지식 모델이 판단과 업무에 쓰이는 자리

원인을 확인할 기준과 실제 값, 조치의 문서 근거까지 연결했습니다. 온톨로지는 이 연결을 같은 뜻으로 정의하는 바탕이죠. 그렇다면 지식을 조회하고 실제 조치로 옮기는 일은 누가 맡을까요?

표에서 누가 다음 행동을 맡는지 따라가 보세요. 정의된 관계를 조회해 원인 후보와 증거를 찾습니다. 에이전트는 실제 값을 대조해 추천안을 만듭니다. 담당자는 근거를 보고 승인하거나 거절합니다.

승인할 때도 현재 상태를 다시 확인합니다. 추천안을 만드는 동안 설비는 계속 움직였으니까요. 거절하면 이 추천안의 명령을 만들지 않고 거절 기록을 남깁니다. 거절 기록 자체가 설비 정지 명령은 아닙니다.

프로세스는 승인된 조치를 명령으로 내려보내고, 실행 뒤에도 재관측합니다. 원하는 완화를 확인하지 못하면 사람에게 다시 넘깁니다. 모델은 판단의 기준을 주고, 에이전트와 사람, 프로세스는 서로 다른 책임을 맡습니다.

[SLIDE 70~71] 실습: 설비 이상과 원인 · 조치의 관계 그리기

판단의 기준이 되는 관계를 직접 표현해 보는 실습입니다. 설비와 센서, 증상, 원인, 조치를 연결한 그림을 만듭니다. 필요한 고장 유형과 구성 요소도 함께 놓고, 각 화살표의 이름과 방향을 정해 보세요. 우선 핀 오염을 판별할 증거와 핀 세척 조치가 연결되게 그립니다.

그림이 완성되면 팬 구동 불량 가지를 더하겠습니다. 같은 냉각 성능 상실이라도 팬에 문제가 있다면 확인할 증거와 손볼 곳이 달라지죠. 매뉴얼에서 팬 진동의 판별 기준과 팬 점검 근거를 찾아 기존 그림에 연결해 보세요.

결과 검토에서는 두 원인이 공유하는 고장 유형과, 서로 다른 증거·조치를 짚겠습니다. 팬 구동 불량도 핀 세척으로 이어 놓았다면 원문을 보고 고쳐야 해요. 마지막에는 완성 그림에서 두 가지 경로를 읽어 보겠습니다. 화살표를 문장으로 풀었을 때, 원인이 달라지면 무엇을 다르게 확인하고 어떻게 조치하는지 설명할 수 있으면 됩니다.

[SLIDE 72] 경보·판단·승인·실행 기록의 구별

오늘 배운 흐름을 쿨러 사건의 기록으로 복습해 보겠습니다. 관계 그림에서 연결한 지식이 실제로 쓰인 기록 다섯 개가 화면에 있습니다. 각각 무엇을 확인하는 기록인지 연결해 보세요. 선택할 것은 경보, 원인 진단, 실행 승인, 명령 처리, 완화 확인입니다. 특히 아래 두 기록이 같은 뜻인지 생각해 보세요.

첫 행은 경보입니다. 유온과 냉각 효율이 조건을 벗어났다는 기록이죠. 둘째는 원인 진단입니다. 평균값을 판별 기준과 대조해 핀 오염을 확인했습니다. 경보가 났다는 것만으로는 이 둘째 결론을 얻을 수 없었어요.

셋째는 실행 승인입니다. 운전원이 추천안을 검토해 실행하도록 결정한 기록입니다. 넷째는 명령 처리예요. PLC가 해당 명령을 처리했고 인터록도 정상이라고 답했습니다.

마지막은 완화 확인입니다. 명령 뒤 유온이 내려가고 경보도 해제됐다는 관측입니다. 넷째가 있어도 다섯째는 없을 수 있습니다. 명령은 처리됐지만 냉각 효과를 얻지 못한 경우겠죠.

그리고 다섯째가 나왔다고 핀 세척이 끝난 것은 아닙니다. 팬으로 유온을 낮췄지만 오염은 남아 있었어요. 근본 조치는 별도 작업지시로 이어졌습니다. 각 기록이 무엇을 확인했는지 구별해야, 남은 일도 놓치지 않을 수 있습니다.

[SLIDE 73] 오늘 다룬 핵심 개념과 주요 용어 정리

사건의 흐름을 되짚었으니, 이번에는 용어의 뜻을 정리하겠습니다. 표에는 약어의 정식 명칭과 풀이가 함께 있습니다.

OT는 설비를 감시하고 제어하는 운영 기술이고, IT는 데이터를 다루는 정보 기술입니다. DMZ는 두 영역 사이에서 연결을 제한하는 완충 구역이에요. PLC는 센서 값을 읽고 정해진 논리로 설비를 직접 제어하는 산업용 제어기입니다.

MQTT는 발행자의 메시지를 브로커가 구독자에게 전달하는 경량 메시지 규약입니다. SCADA는 설비 데이터를 모아 감시하고 원격으로 제어하는 시스템이고, HMI는 사람이 설비 상태를 보고 조작하는 화면이에요.

HITL은 자동 처리 중간에 사람이 승인이나 거절 같은 판단을 하는 구조입니다. MES는 생산 지시와 실적, 납기를 관리하는 제조 실행 시스템입니다.

[SLIDE 74] Review Quiz: 핵심 흐름 되짚어보기

배운 내용을 스스로 판단에 쓸 수 있는지, 퀴즈 세 문제로 확인해 보겠습니다. 문제마다 보기를 읽은 뒤 잠시 멈추고 답을 골라 보세요. 정답과 까닭은 세 문제를 모두 읽은 뒤 확인합니다.

[SLIDE 75] Quiz 1. 경보 이후의 판단

첫 번째 문제입니다. 쿨러 성능 저하 경보가 났지만, 이번에는 팬 진동이 앞 사건과 다릅니다. 팬 진동의 평균은 초당 1.1밀리미터이고, 캐비닛 온도는 28도입니다.

보기는 네 가지입니다. 경보 이름만으로 핀 오염을 정한다, 진동을 근거로 팬 구동 불량을 의심한다, 원인과 관계없이 팬을 올린다, 캐비닛 온도만 보고 이상이 없다고 판단한다. 판별 기준과 주어진 값을 대조해서, 내릴 수 있는 판단을 골라 보세요.

[SLIDE 76] Quiz 2. 관계의 이름과 방향

두 번째는 관계의 이름과 방향을 판단하는 문제입니다. 핀 오염이라는 원인과 냉각 성능 상실이라는 고장 유형을 연결합니다.

첫 보기는 냉각 성능 상실이 핀 오염을 일으킨다는 연결입니다. 둘째는 핀 오염이 냉각 성능 상실을 가리킨다는 연결이고, 셋째는 핀 오염이 냉각 성능 상실을 일으킨다는 연결입니다. 넷째는 이름과 방향 없이 선만 잇자는 것입니다. 네 보기의 문장 뜻과 양 끝 대상의 종류를 함께 보고, 맞는 연결을 골라 보세요.

[SLIDE 77] Quiz 3. 추천과 실행의 책임 경계

마지막은 추천과 실행을 구별하는 문제입니다. 에이전트가 핀 오염으로 판단하고 팬 최대 운전을 추천했습니다. 담당자는 아직 승인하지 않았습니다. 지금 설비 명령은 어떤 상태일까요?

추천과 동시에 전송됐는지, 그래프에 연결돼 있어서 실행됐는지, 시간이 지나면 저절로 실행되는지, 아니면 아직 명령을 만들지 않고 승인을 기다리는지 골라 보세요.

[SLIDE 78] 정답과 판단 기준

고른 답과 그 근거를 함께 대조해 보겠습니다. 첫 문제의 정답은 2번입니다. 이번에는 팬 진동 평균이 팬 구동 불량의 판별 기준 이상입니다. 경보 이름만 보고 핀 오염이라고 정할 수 없어요. 같은 경보여도 확인한 증거가 달라지면 원인에 대한 판단도 달라집니다.

가장 조심할 보기는 원인과 관계없이 팬을 최대로 돌린다는 3번입니다. 팬이 이상한 상태라면 속도를 올려 진동 위험을 키울 수 있었습니다. 캐비닛 온도가 정상이라는 4번도 설비 전체가 정상이라는 뜻은 아니에요. 주변 온도 상승이라는 후보를 가릴 근거 하나일 뿐입니다.

두 번째 정답은 3번입니다. '핀 오염은 냉각 성능 상실을 일으킨다'가 사실과 관계 정의에 맞습니다. 1번은 원인과 결과를 뒤집었어요. 2번의 '가리킨다'는 증상에서 고장 유형으로 잇는 관계인데 원인을 넣었습니다. 자연스러운 문장처럼 보여도 양 끝의 종류가 어긋납니다. 이름과 방향을 빼면 이 차이를 표현할 수 없어요.

세 번째 정답은 4번입니다. 추천은 나왔지만 승인 전이므로 설비 명령은 아직 없습니다. 그래프에서 조치를 찾았다는 것과 실행하도록 승인했다는 것은 다릅니다. 오래 기다렸다고 사람이 승인하는 단계를 대신할 수는 없습니다.

여기까지가 이번 회차입니다.

다음 회차에서는 설비와 센서, 원인과 조치의 관계를 어떻게 정리할지 배우겠습니다. 그 지식을 그래프에 담고, 필요한 정보를 찾아보는 것까지 직접 해 볼 거예요. 오늘 수고 많으셨습니다. 감사합니다.